Assignment 4: Exploring Instruction-Level Parallelism (ILP) in Modern Processors

Vishnu Sahith Jella

University of the Cumberlands

Computer Architecture and Design (MSCS-531-M40)

Dr. Vanessa Cooper

18th Oct 2026

Part 1: Understanding Instruction-Level Parallelism

1. Introduction and Evolution of ILP

Instruction Level Parallelism (ILP) refers to the processor's capacity to overlap or execute many separate instructions from a single instruction stream. The concept grew as processor designers moved beyond purely sequential execution. The first major stride was pipelining, which allowed distinct instructions to occupy the fetch, decode, execute, memory, and writeback stages simultaneously. Hardware might detect ready instructions at runtime, handle changing execution latency, and decrease false dependencies by register renaming, as later demonstrated by scoreboard and Tomasulo-style dynamic scheduling. This idea was taken further by superscalar processors, which fetched, decoded, issued and completed more than one instruction every cycle. Modern out-of-order processors leverage wide issue, speculative execution, dynamic scheduling, register renaming, branch prediction, memory disambiguation, and huge instruction windows to expose as much meaningful parallelism as feasible (Hennessy & Patterson, 2019).

Thus, the historical trend in ILP has shifted from fixed overlap to increasingly dynamic exploitation of independence. Early pipelines were strongly dependent on compiler scheduling , and halted whenever risks occurred. Instead, modern processors look beyond a blocked instruction to find later independent work and resume execution. This helped performance significantly, but it also complicated the processor front end, scheduling logic, reorder buffers, branch predictors, and memory systems. The current research trend is shifting from simply increasing instruction window sizes to efficiently exploiting ILP with less energy and hardware overhead.

2. Core Concepts: Detecting and Exploiting Parallelism

A modern processor detects ILP by tracking instruction dependencies and separating true dependencies from artificial ones. Read-after-write relationships are true data dependencies and must be preserved. Write-after-read and write-after-write relationships can often be removed through register renaming. Once instructions are renamed, scheduling hardware can issue operations when their source operands and required functional units are available, even if older unrelated instructions are still waiting. This is the foundation of out-of-order execution. The reorder buffer then restores precise architectural state by committing completed instructions in program order.

Control-flow prediction is equally important. Wide issue is useful only if the front end can continuously supply enough correct-path instructions. A branch misprediction interrupts that supply, discards speculative work, and reduces the effective instruction window. The 2026 HOTP study by Harsh and Renau emphasizes the tension between branch-prediction latency and accuracy in wide superscalar machines. Their multi-level design combines a fast, high-confidence taken predictor with larger TAGE-based components and reports up to a 3.7% IPC improvement over a conventional three-level design. The significance is not just the percentage gain; it shows that

branch prediction remains a first-order ILP problem because a very wide back end cannot exploit instructions the front end fails to deliver accurately (Harsh & Renau, 2026).

Instruction supply can also be limited by code layout and instruction-cache behavior. Zhang et al. (2022) identify the processor front end as an increasing bottleneck for large data-center applications. Their OCOLOS system performs online code layout optimization, arranging frequently executed code to improve instruction-cache locality and branch-prediction effectiveness. OCOLOS reports speedups of up to 1.41x for MySQL, 2.20x for Verilator, and 1.14x for a Clang build. This work broadens the idea of ILP: exploiting parallel execution is not only a scheduling problem inside the core; it also depends on feeding the core a timely and predictable instruction stream.

3. Fundamental Limitations of ILP

The most important limit on ILP is the dependency structure of the program itself. True data dependencies form chains that prediction or algorithmic transformation cannot break. A second limit is control dependence. When the next instruction address depends on a branch outcome, processors must either wait or speculate. A third limit comes from memory dependence: loads and stores may reference addresses that are not known early enough to prove independence. Finally, finite resources limit how much parallelism we can extract. Issue queues, reorder buffers, physical registers, load/store queues, functional units, cache ports, and memory bandwidth are all bounded.

These limits help explain why simply increasing issue width produces diminishing returns. A wider processor needs more instructions ready at the same time, more register-file bandwidth, more bypass paths, more wakeup/select logic, and more front-end throughput. The hardware cost can grow faster than the performance gain. Power is also critical because speculative instructions and wrong-path execution consume energy even when they do not contribute to committed work. As a result, current research increasingly seeks complexity-effective ILP instead of maximum ILP at any cost.

4. Contemporary Research: Complexity-Effective ILP

Two recent GPU studies show that ILP remains valuable even in architectures traditionally designed around thread-level parallelism. Matsuo et al. (2024) propose TURBULENCE, a low-cost out-of-order GPU architecture based on a distance-based ISA. Instead of naming operands with conventional register identifiers, the ISA refers to operands by inter-instruction distance. This reduces false dependencies and lets the design avoid expensive structures normally associated with out-of-order execution, including complex renaming and load/store machinery. The reported result is a 17.6% performance improvement with no increase in energy consumption over the evaluated baseline. The study matters because it shows that ILP research is no longer limited to conventional CPUs; designers are adapting ILP mechanisms for throughput-oriented processors while deliberately controlling complexity.

Chaturvedi et al. (2024) reach a related conclusion with GhOST, a lightweight GPU out-of-order scheduling technique. GhOST reuses information already present in decode and issue structures rather than adding a conventional full out-of-order engine. Across the evaluated GPU binaries, the paper reports a 6.9% geometric-mean speedup, a maximum improvement of 36%, and only 0.007% additional area. The authors also show that results can depend strongly on the actual final binary because compiler scheduling may already remove opportunities that appear in intermediate representations. This observation is particularly relevant to ILP evaluation: the measured opportunity for dynamic scheduling depends on the compiler, the instruction set, and the final code stream, not just on the source-level algorithm.

5. Memory Latency, Runahead, and Complementary Parallelism

Long-latency memory accesses can render an otherwise wide out-of-order processor ineffective because dependent instructions pile up behind cache misses. Naithani et al. (2023) address this problem with Decoupled Vector Runahead (DVR). DVR creates a lightweight speculative subthread that follows indirect memory-access chains and exposes memory-level parallelism ahead of the main thread. The paper reports substantial gains on graph analytics, database, and high-performance computing workloads, including performance 2.4x higher than a large five-wide out-of-order baseline on the evaluated set. This work illustrates a major direction in modern ILP research: instead of making the main scheduler indefinitely larger, designers can use specialized speculative mechanisms to generate future work and overlap expensive memory latency.

The connection between ILP and memory-level parallelism is important. A processor may have independent arithmetic instructions available, but if memory delays the values they need, the useful instruction window shrinks. Techniques such as runahead execution, prefetching, memory dependence prediction, and larger nonblocking cache structures therefore complement ILP. The most successful modern designs do not treat branch prediction, cache behavior, memory parallelism, and instruction scheduling as independent topics; they optimize them as a coordinated pipeline.

6. Performance Metrics and Design Trade-offs

IPC is one of the most common measures of ILP effectiveness because it directly expresses how many instructions a processor completes per cycle. CPI is its reciprocal for a fixed instruction stream and expresses the cycle cost per committed instruction. However, neither metric should be interpreted without context. A processor can increase IPC by using more hardware, consuming more power, or speculating more aggressively. Latency matters for individual tasks, while throughput matters when many instructions or threads must be completed. Energy per instruction, area, branch mispredictions, cache misses, and functional-unit utilization are therefore important complementary measures.

The contemporary papers illustrate these trade-offs clearly. HOTP trades predictor organization and latency against accuracy; OCOLOS improves instruction delivery partly through software

layout rather than wider hardware; TURBULENCE and GhOST seek out-of-order benefits while minimizing hardware cost; and DVR adds a specialized speculative mechanism to tolerate memory stalls. Collectively, these studies suggest that future ILP gains are more likely to come from targeted mechanisms and hardware/software co-design than from simply scaling traditional superscalar structures.

7. Current Challenges and Future Directions

Current ILP design faces three related challenges: diminishing returns, power limits, and workload diversity. As issue width and instruction windows grow, dependence chains and branch behavior prevent proportional performance scaling. Larger scheduling structures are expensive to search every cycle and increase wire delay and energy. At the same time, modern systems run data-center services, machine-learning frameworks, graphics workloads, and irregular graph applications that stress different parts of the pipeline. A single aggressive core configuration is therefore unlikely to be optimal across all workloads.

The most promising future directions are heterogeneous and adaptive. Lightweight out-of-order techniques for GPUs, specialized runahead engines, and online code optimization already show how ILP can be combined with thread-level, memory-level, and domain-specific parallelism. Machine-learning-guided prediction and automated design-space search may improve branch predictors, prefetchers, and resource allocation. Heterogeneous systems may use different core types so that highly serial code runs on strong ILP-oriented cores while throughput-oriented code moves to many-core or accelerator structures. The broader trend is to extract the right form of parallelism for each execution phase rather than rely on a single mechanism.

In summary, ILP remains a central performance mechanism, but efficiency, not unlimited width, will shape its future. The literature from 2022-2026 shows active research in front-end optimization, branch prediction, memory-latency tolerance, and low-cost out-of-order execution. These approaches preserve the original ILP goal - finding independent work inside a sequential instruction stream - while adapting it to modern constraints in power, complexity, and heterogeneous computing.

Part 2: Practical Exploration of ILP Techniques in gem5

8. Experimental Environment and Workload

The practical experiments used gem5 25.1.0.1 with the x86 ISA under Ubuntu in Windows Subsystem for Linux. The existing Hello World executable and Python configuration from the earlier assignments were reused so that the architectural configuration, rather than the application source, was the main experimental variable. The initial verification confirmed the gem5 executable, run_hello.py, and the compiled hello program were present and that the program ran natively.

```
vishnusahithjella@DESKTOP-C34OBRR:~$ cd gem5/
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ build/X86/gem5.opt --version 2>/dev/null || build/X86/gem5.opt --help | head -5
Usage
=====
  gem5.opt [gem5 options] script.py [script options]

gem5 is copyrighted software; use the --copyright option for details.
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ ls -lh hello run_hello.py build/X86/gem5.opt
-rwxr-xr-x 1 vishnusahithjella vishnusahithjella 941M Oct  4 09:52 build/X86/gem5.opt
-rwxr-xr-x 1 vishnusahithjella vishnusahithjella  16K Sep 20 11:09 hello
-rw-r--r-- 1 vishnusahithjella vishnusahithjella 4.7K Sep 20 11:29 run_hello.py
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ ./hello
Hello, World!
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

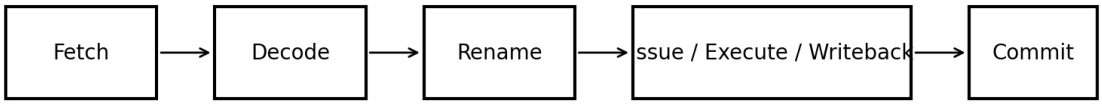
Figure 1. Verification of the gem5 environment and Hello World workload.

9. Basic O3 Pipeline Simulation

A new configuration file, run_ilp.py, was created from run_hello.py, and the CPU model was changed from X86TimingSimpleCPU to X86O3CPU. The O3 model is appropriate for ILP studies because it models dynamic scheduling, speculation, and out-of-order execution. Its internal stages are more detailed than a textbook five-stage pipeline, but they map to the same basic functions: instruction fetch, decode, dependency handling/renaming, execution and writeback, and in-order commit.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ cp run_hello.py run_ilp.py && sed -i 's/X86TimingSimpleCPU()/X86O3CPU()/' run_ilp.py && gre
p -n "CPU()" run_ilp.py
64:system.cpu = X86O3CPU()
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 2. run_ilp.py configured to use X86O3CPU.



gem5 X86O3CPU stage mapping used for ILP analysis

Figure 3. Conceptual mapping of the gem5 O3 pipeline stages used in this analysis.

An initial attempt to force fetch, decode, rename, dispatch, issue, writeback, and commit widths all to one instruction per cycle caused a gem5 TimeBuffer assertion during execution. The failure demonstrated that aggressively constraining all O3 stage widths was not a stable configuration for this gem5 build. We removed the manual width settings, after which the default X86O3CPU configuration executed the workload successfully.

```
/usr/lib/x86_64-linux-gnu/libc.so.6(pthread_kill+0x11c) [0x705a44aa61ac]
/usr/lib/x86_64-linux-gnu/libc.so.6(gsignal+0x1e) [0x705a44a45b7e]
/usr/lib/x86_64-linux-gnu/libc.so.6(abort+0x27) [0x705a44a288ec]
/usr/lib/x86_64-linux-gnu/libc.so.6(+0x29979) [0x705a44a29979]
/usr/lib/x86_64-linux-gnu/libc.so.6(__assert_fail+0xb5) [0x705a44a3bf75]
build/X86/gem5.opt(+0x8b3a09) [0x55f12eb03a09]
build/X86/gem5.opt(_ZN4gem52o33IEW12executeInstsEv+0x9be) [0x55f12f52cf6e]
build/X86/gem5.opt(_ZN4gem52o33IEW4tickEv+0x6e8) [0x55f12f531f48]
build/X86/gem5.opt(_ZN4gem52o33CPU4tickEv+0x153) [0x55f12f4f0c13]
build/X86/gem5.opt(_ZN4gem510EventQueue10serviceOneEv+0xb5) [0x55f130b107e5]
build/X86/gem5.opt(_ZN4gem59doSimLoopEPNS_10EventQueueE+0x68) [0x55f130b42718]
build/X86/gem5.opt(_ZN4gem58simulateEm+0x26b) [0x55f130b42e0b]
build/X86/gem5.opt(+0x185fadf) [0x55f12faafadf]
build/X86/gem5.opt(+0xc24142) [0x55f12ee74142]
/usr/lib/x86_64-linux-gnu/libpython3.14.so.1.0(+0x18d105) [0x705a4598d105]
/usr/lib/x86_64-linux-gnu/libpython3.14.so.1.0(PyObject_Call+0xab) [0x705a459271bb]
/usr/lib/x86_64-linux-gnu/libpython3.14.so.1.0(_PyEval_EvalFrameDefault+0x681f) [0x705a45a8725f]
/usr/lib/x86_64-linux-gnu/libpython3.14.so.1.0(+0x2941a5) [0x705a45a941a5]
/usr/lib/x86_64-linux-gnu/libpython3.14.so.1.0(PyEval_EvalCode+0xd6) [0x705a45a791c6]
/usr/lib/x86_64-linux-gnu/libpython3.14.so.1.0(+0x27a819) [0x705a45a7a819]
/usr/lib/x86_64-linux-gnu/libpython3.14.so.1.0(PyObject_Vectorcall+0x5e) [0x705a45924cde]
/usr/lib/x86_64-linux-gnu/libpython3.14.so.1.0(_PyEval_EvalFrameDefault+0x4fa0) [0x705a45a859e0]
/usr/lib/x86_64-linux-gnu/libpython3.14.so.1.0(+0x2940f7) [0x705a45a940f7]
build/X86/gem5.opt(+0x183b966) [0x55f12fa8b966]
build/X86/gem5.opt(main+0x217) [0x55f12ee5d737]
/usr/lib/x86_64-linux-gnu/libc.so.6(+0x2a601) [0x705a44a2a601]
/usr/lib/x86_64-linux-gnu/libc.so.6(__libc_start_main+0x88) [0x705a44a2a718]
build/X86/gem5.opt(_start+0x25) [0x55f12ee60475]
--- END LIBC BACKTRACE ---
For more info on how to address this issue, please visit https://www.gem5.org/documentation/general_docs/common-errors/

Aborted                    build/X86/gem5.opt --outdir=m5out_ilp_single run_ilp.py
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 4. Troubleshooting evidence: O3 TimeBuffer assertion after forcing all stage widths to one.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ sed -i '/system.cpu.fetchWidth = 1/d;/system.cpu.decodeWidth = 1/d;/system.cpu.renameWidth
= 1/d;/system.cpu.dispatchWidth = 1/d;/system.cpu.issueWidth = 1/d;/system.cpu.wbWidth = 1/d;/system.cpu.commitWidth = 1/d' run_ilp.p
y && rm -rf m5out_ilp_o3 && build/X86/gem5.opt --outdir=m5out_ilp_o3 run_ilp.py
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 25.1.0.1
gem5 compiled Oct  4 2026 09:48:48
gem5 started Oct 18 2026 11:33:13
gem5 executing on DESKTOP-C34OBRR, pid 645
command line: build/X86/gem5.opt --outdir=m5out_ilp_o3 run_ilp.py

Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Gr
oup. Legacy stat is deprecated.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:32] into equal-sized buckets. Rounding up.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:8] into equal-sized buckets. Rounding up.
system.remote_gdb: Listening for connections on port 7000
Beginning simulation!
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
src/sim/syscall_emul.cc:86: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall rseq(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
Hello, World!
Exiting @ tick 884709000 because exiting with last active thread context
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 5. Successful baseline X86O3CPU simulation.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ grep -E "simInsts|numCycles|\.ipc|\.cpi" m5out_ilp_o3/stats.txt | head -20
simInsts                                     99526                       # Number of instructions simulated (Count)
system.cpu.numCycles                        884710                       # Number of cpu cycles simulated (Cycle)
system.cpu.cpi                            8.889235                       # CPI: cycles per instruction (core level) ((Cycle/Count)
)
system.cpu.ipc                            0.112496                       # IPC: instructions per cycle (core level) ((Count/Cycle)
)
system.cpu.commitStats0.cpi               8.889235                       # CPI: cycles per instruction (thread level) ((Cycle/Coun
t))
system.cpu.commitStats0.ipc               0.112496                       # IPC: instructions per cycle (thread level) ((Count/Cycl
e))
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 6. Baseline O3 instruction count, cycles, CPI, and IPC.

Baseline O3 Metric	Value
Simulated instructions	99,526
CPU cycles	884,710
CPI	8.889235
IPC / instruction throughput	0.112496
Exit tick	884,709,000

IPC was used as the primary instruction-throughput measure. CPI is reported as a cycle-cost metric for the workload. A direct average per-instruction pipeline latency statistic was not collected by the selected configuration, so CPI is not presented as if it were an exact instruction-latency measurement.

10. Impact of Branch Prediction

The assignment asks for branch-prediction experimentation. In this gem5 O3 configuration, completely disabling the predictor is not a clean comparison because the O3 front end is designed around speculative branch prediction. We therefore performed a controlled predictor comparison: we compared the default O3 predictor with LocalBP while keeping the workload and O3 processor model unchanged.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ cp run_ilp.py run_ilp_localbp.py && sed -i '65i system.cpu.branchPred = BranchPredictor(con
ditionalBranchPred=LocalBP(), btb=SimpleBTB(), numThreads=1)' run_ilp_localbp.py && grep -n "branchPred" run_ilp_localbp.py
65:system.cpu.branchPred = BranchPredictor(conditionalBranchPred=LocalBP(), btb=SimpleBTB(), numThreads=1)
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 7. LocalBP branch predictor configuration.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ rm -rf m5out_ilp_localbp && build/X86/gem5.opt --outdir=m5out_ilp_localbp run_ilp_localbp.p
y
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 25.1.0.1
gem5 compiled Oct  4 2026 09:48:48
gem5 started Oct 18 2026 11:36:05
gem5 executing on DESKTOP-C34OBRR, pid 682
command line: build/X86/gem5.opt --outdir=m5out_ilp_localbp run_ilp_localbp.py

Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Gr
oup. Legacy stat is deprecated.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:32] into equal-sized buckets. Rounding up.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:8] into equal-sized buckets. Rounding up.
system.remote_gdb: Listening for connections on port 7000
Beginning simulation!
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
src/sim/syscall_emul.cc:86: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall rseq(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
Hello, World!
Exiting @ tick 876079000 because exiting with last active thread context
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 8. Successful LocalBP simulation.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ grep -Ei "simInsts|numCycles|\.ipc|\.cpi|branchPred.*(mispred|miss|lookups)|condPredicted|c
ondIncorrect" m5out_ilp_localbp/stats.txt | head -40
simInsts                                    99526                       # Number of instructions simulated (Count)
system.cpu.numCycles                       876080                       # Number of cpu cycles simulated (Cycle)
system.cpu.cpi                            8.802524                      # CPI: cycles per instruction (core level) ((Cycle/Count)
)
system.cpu.ipc                            0.113604                      # IPC: instructions per cycle (core level) ((Count/Cycle)
)
system.cpu.branchPred.lookups_0::NoBranch         1357      3.66%      3.66% # Number of BP lookups (Count)
system.cpu.branchPred.lookups_0::Return          1135      3.06%      6.72% # Number of BP lookups (Count)
system.cpu.branchPred.lookups_0::CallDirect         1124      3.03%      9.76% # Number of BP lookups (Count)
system.cpu.branchPred.lookups_0::CallIndirect          171      0.46%     10.22% # Number of BP lookups (Count)
system.cpu.branchPred.lookups_0::DirectCond        29210     78.81%     89.02% # Number of BP lookups (Count)
system.cpu.branchPred.lookups_0::DirectUncond         3699      9.98%     99.00% # Number of BP lookups (Count)
system.cpu.branchPred.lookups_0::IndirectCond            0      0.00%     99.00% # Number of BP lookups (Count)
system.cpu.branchPred.lookups_0::IndirectUncond          369      1.00%    100.00% # Number of BP lookups (Count)
system.cpu.branchPred.lookups_0::total          37065                       # Number of BP lookups (Count)
system.cpu.branchPred.squashes_0::NoBranch         1291      8.04%      8.04% # Number of branches that got squashed (completely remo
ved) as an earlier branch was mispredicted. (Count)
system.cpu.branchPred.squashes_0::Return          285      1.77%      9.81% # Number of branches that got squashed (completely remove
d) as an earlier branch was mispredicted. (Count)
system.cpu.branchPred.squashes_0::CallDirect          380      2.37%     12.18% # Number of branches that got squashed (completely re
moved) as an earlier branch was mispredicted. (Count)
system.cpu.branchPred.squashes_0::CallIndirect           60      0.37%     12.55% # Number of branches that got squashed (completely
removed) as an earlier branch was mispredicted. (Count)
system.cpu.branchPred.squashes_0::DirectCond        12072     75.15%     87.70% # Number of branches that got squashed (completely re
moved) as an earlier branch was mispredicted. (Count)
system.cpu.branchPred.squashes_0::DirectUncond         1830     11.39%     99.10% # Number of branches that got squashed (completely
removed) as an earlier branch was mispredicted. (Count)
system.cpu.branchPred.squashes_0::IndirectCond            0      0.00%     99.10% # Number of branches that got squashed (completely
removed) as an earlier branch was mispredicted. (Count)
system.cpu.branchPred.squashes_0::IndirectUncond          145      0.90%    100.00% # Number of branches that got squashed (completel
y removed) as an earlier branch was mispredicted. (Count)
```

Figure 9. LocalBP performance and branch-predictor statistics.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ grep -E "branchPred\.lookups_0::total|branchPred\.mispredicted_0::total" m5out_ilp_o3/stats
.txt
system.cpu.branchPred.lookups_0::total          35162                       # Number of BP lookups (Count)
system.cpu.branchPred.mispredicted_0::total         2861                       # Number of committed branches that were mispredicted.
 (Count)
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 10. Default O3 branch lookups and committed mispredictions.

Metric	Default O3	LocalBP	Observation
CPU cycles	884,710	876,080	LocalBP used about 0.98% fewer cycles
CPI	8.889235	8.802524	Lower with LocalBP
IPC	0.112496	0.113604	About 0.98% higher with LocalBP
Branch lookups	35,162	37,065	Different predictor behavior changes lookup population
Committed mispredictions	2,861	2,777	84 fewer with LocalBP
Misprediction rate	8.14%	7.49%	Lower normalized rate with LocalBP

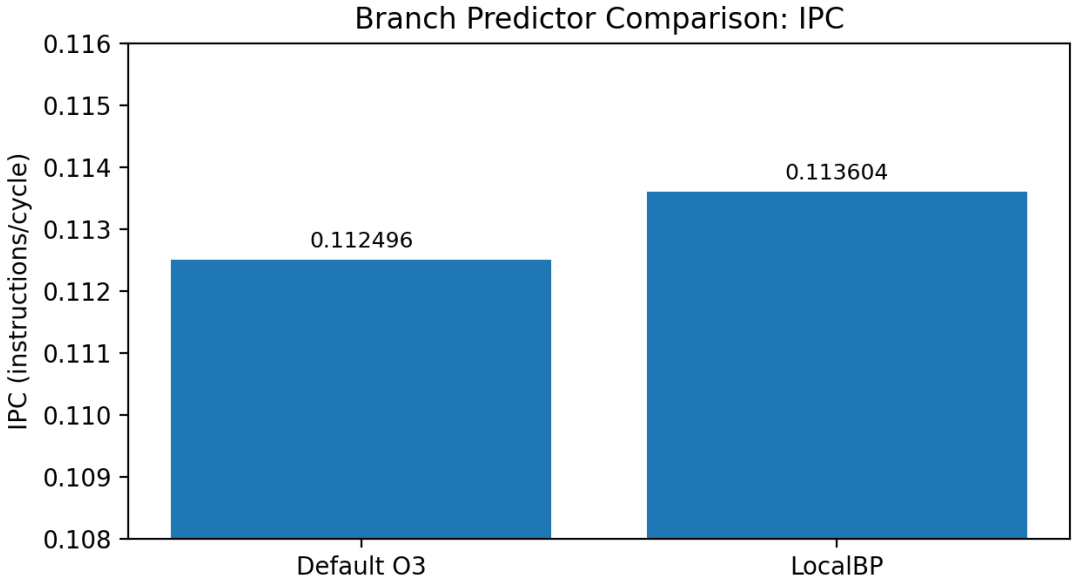


Figure 11. IPC comparison for the default O3 predictor and LocalBP.

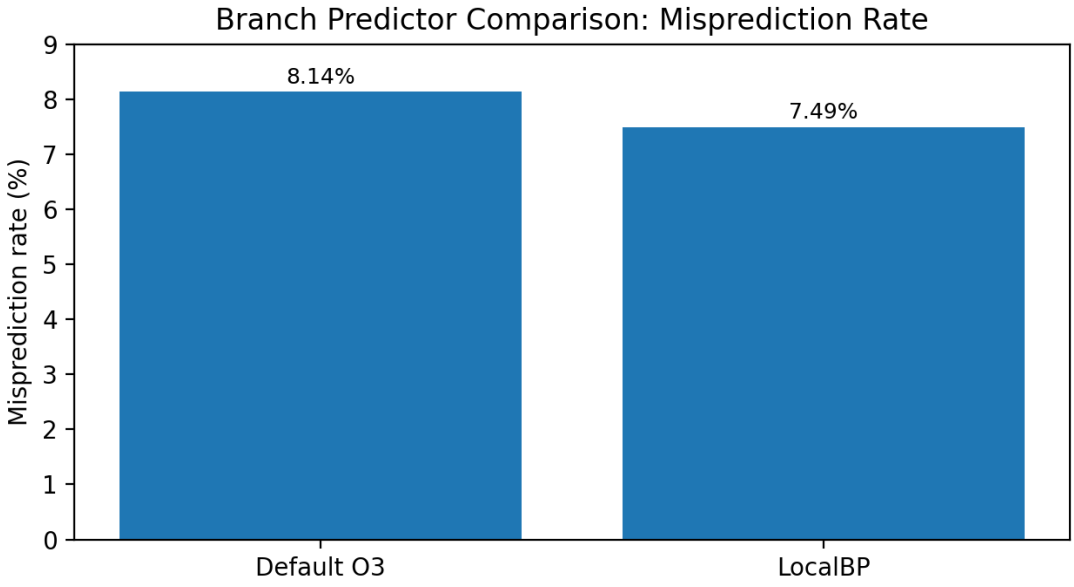


Figure 12. Committed branch misprediction rate for the two predictor configurations.

For this small workload, LocalBP produced a modest but measurable improvement: IPC rose from 0.112496 to 0.113604 while the normalized committed misprediction rate fell from about 8.14% to 7.49%. The result supports the general ILP principle that more accurate control-flow prediction keeps the pipeline supplied with useful instructions and reduces work lost to recovery.

11. Multiple-Issue / Superscalar Experiment

To study multiple issue without repeating the earlier unstable all-stage width restriction, we constrained issue Width to one instruction per cycle. This created a single-issue bottleneck inside

the otherwise normal O3 processor. The result was then compared with the normal multi-issue O3 configuration.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ cp run_ilp.py run_ilp_issue1.py && sed -i '65i system.cpu.issueWidth = 1' run_ilp_issue1.py
 && grep -n -E "X86O3CPU|issueWidth" run_ilp_issue1.py
64:system.cpu = X86O3CPU()
65:system.cpu.issueWidth = 1
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 13. O3 configuration with issueWidth limited to one.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ rm -rf m5out_ilp_issue1 && build/X86/gem5.opt --outdir=m5out_ilp_issue1 run_ilp_issue1.py
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 25.1.0.1
gem5 compiled Oct  4 2026 09:48:48
gem5 started Oct 18 2026 11:41:08
gem5 executing on DESKTOP-C34OBRR, pid 965
command line: build/X86/gem5.opt --outdir=m5out_ilp_issue1 run_ilp_issue1.py

Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Gr
oup. Legacy stat is deprecated.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:32] into equal-sized buckets. Rounding up.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:8] into equal-sized buckets. Rounding up.
system.remote_gdb: Listening for connections on port 7000
Beginning simulation!
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
src/sim/syscall_emul.cc:86: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall rseq(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
Hello, World!
Exiting @ tick 905609000 because exiting with last active thread context
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 14. Successful issueWidth = 1 simulation.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ grep -E "simInsts|numCycles|\.ipc|\.cpi" m5out_ilp_issue1/stats.txt | head -20
simInsts                                     99526                       # Number of instructions simulated (Count)
system.cpu.numCycles                        905610                       # Number of cpu cycles simulated (Cycle)
system.cpu.cpi                            9.099230                       # CPI: cycles per instruction (core level) ((Cycle/Count)
)
system.cpu.ipc                            0.109899                       # IPC: instructions per cycle (core level) ((Count/Cycle)
)
system.cpu.commitStats0.cpi               9.099230                       # CPI: cycles per instruction (thread level) ((Cycle/Coun
t))
system.cpu.commitStats0.ipc               0.109899                       # IPC: instructions per cycle (thread level) ((Count/Cycl
e))
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 15. Performance statistics for the single-issue-constrained run.

Metric	Issue width = 1	Normal O3 multi-issue	Difference
Instructions	99,526	99,526	Same workload
CPU cycles	905,610	884,710	Normal O3 uses 20,900 fewer cycles
CPI	9.099230	8.889235	Lower with multi-issue
IPC	0.109899	0.112496	Multi-issue about 2.36% higher

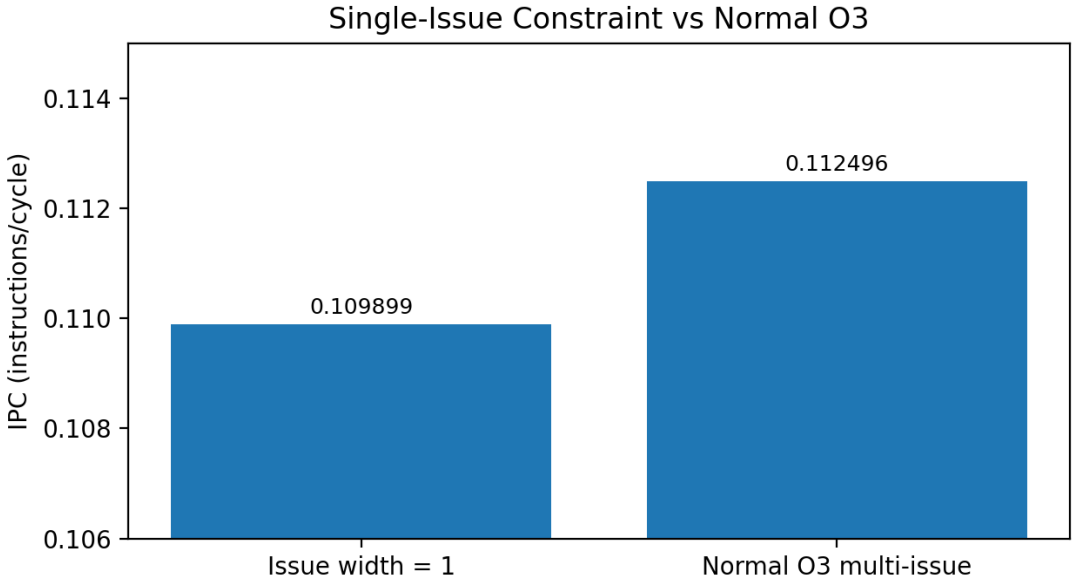


Figure 16. IPC improvement from normal multi-issue O3 execution.

The experiment shows a small but clear superscalar benefit on this workload. Normal O3 execution achieved approximately 2.36% higher IPC than the issueWidth = 1 configuration. The improvement is limited by the small Hello World workload, which does not contain enough sustained independent computation to fully exercise a wide superscalar core. A larger benchmark suite with integer, floating-point, and memory-intensive programs would likely reveal a wider range of ILP behavior.

12. Simultaneous Multithreading (SMT)

SMT was the most challenging experiment because an x86 multithreaded system requires more than simply setting numThreads = 2. The first custom configuration failed because the system itself was not marked multithreaded. After setting system.multi_thread = True, gem5 reported that the second x86 interrupt port was not connected. These failures were useful troubleshooting evidence because they identified concrete architectural resources that must be replicated or connected for the second hardware thread.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ rm -rf m5out_ilp_smt && build/X86/gem5.opt --outdir=m5out_ilp_smt run_ilp_smt.py
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 25.1.0.1
gem5 compiled Oct  4 2026 09:48:48
gem5 started Oct 18 2026 11:46:04
gem5 executing on DESKTOP-C34OBRR, pid 1014
command line: build/X86/gem5.opt --outdir=m5out_ilp_smt run_ilp_smt.py

Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Gr
oup. Legacy stat is deprecated.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:32] into equal-sized buckets. Rounding up.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:8] into equal-sized buckets. Rounding up.
gem5.opt: src/cpu/base.cc:495: void gem5::BaseCPU::registerThreadContexts(): Assertion `system->multiThread || numThreads == 1' faile
d.
Program aborted at tick 0
--- BEGIN LIBC BACKTRACE ---
build/X86/gem5.opt(_ZN4gem515print_backtraceEv+0x2b) [0x5f1938c7bb7b]
build/X86/gem5.opt(_ZN4gem512abortHandlerEi+0x4c) [0x5f1938ca50bc]
/usr/lib/x86_64-linux-gnu/libc.so.6(+0x45cb0) [0x77dee1645cb0]
/usr/lib/x86_64-linux-gnu/libc.so.6(pthread_kill+0x11c) [0x77dee16a61ac]
/usr/lib/x86_64-linux-gnu/libc.so.6(gsignal+0x1e) [0x77dee1645b7e]
/usr/lib/x86_64-linux-gnu/libc.so.6(abort+0x27) [0x77dee16288ec]
/usr/lib/x86_64-linux-gnu/libc.so.6(+0x29979) [0x77dee1629979]
/usr/lib/x86_64-linux-gnu/libc.so.6(__assert_fail+0xb5) [0x77dee163bf75]
build/X86/gem5.opt(+0x89f403) [0x5f1936c73403]
build/X86/gem5.opt(_ZN4gem57BaseCPU4initEv+0xf0) [0x5f19375ee400]
build/X86/gem5.opt(_ZN4gem52o33CPU4initEv+0xd) [0x5f193767a66d]
build/X86/gem5.opt(+0x187dda2) [0x5f1937c51da2]
```

Figure 17. Initial SMT failure showing the system multi-thread requirement.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ rm -rf m5out_ilp_smt && build/X86/gem5.opt --outdir=m5out_ilp_smt run_ilp_smt.py
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 25.1.0.1
gem5 compiled Oct  4 2026 09:48:48
gem5 started Oct 18 2026 11:47:49
gem5 executing on DESKTOP-C34OBRR, pid 1037
command line: build/X86/gem5.opt --outdir=m5out_ilp_smt run_ilp_smt.py

Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Gr
oup. Legacy stat is deprecated.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:32] into equal-sized buckets. Rounding up.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:8] into equal-sized buckets. Rounding up.
system.remote_gdb: Listening for connections on port 7000
src/arch/x86/interrupts.cc:330: panic: panic condition !intRequestPort.isConnected() occurred: Int port not connected to anything!
Memory Usage: 659960 KBytes
Program aborted at tick 0
--- BEGIN LIBC BACKTRACE ---
build/X86/gem5.opt(_ZN4gem515print_backtraceEv+0x2b) [0x61c79f013b7b]
build/X86/gem5.opt(_ZN4gem512abortHandlerEi+0x4c) [0x61c79f03d0bc]
/usr/lib/x86_64-linux-gnu/libc.so.6(+0x45cb0) [0x7d48064a5cb0]
/usr/lib/x86_64-linux-gnu/libc.so.6(pthread_kill+0x11c) [0x7d48064a61ac]
/usr/lib/x86_64-linux-gnu/libc.so.6(gsignal+0x1e) [0x7d48064a5b7e]
/usr/lib/x86_64-linux-gnu/libc.so.6(abort+0x27) [0x7d48064288ec]
build/X86/gem5.opt(+0xc44c85) [0x61c79d3b0c85]
build/X86/gem5.opt(_ZN4gem56X86ISA10Interrupts4initEv+0x190) [0x61c79e83bfd0]
build/X86/gem5.opt(+0x187dda2) [0x61c79dfe9da2]
build/X86/gem5.opt(+0xc24142) [0x61c79d390142]
/usr/lib/x86_64-linux-gnu/libpython3.14.so.1.0(+0x18d105) [0x7d480738d105]
```

Figure 18. Second SMT failure showing the unconnected interrupt-port requirement.

The standard se.py SMT path was then used. A dynamically linked Hello World workload initialized both threads but did not terminate in a practical amount of time. To avoid dynamic-loader and shared-library overhead in syscall-emulation mode, a statically linked executable was built with gcc -static. The static program executed normally outside gem5 and was then used for both SMT threads.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ gcc -static hello.c -o hello_static && file hello_static && ./hello_static
hello_static: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, BuildID[sha1]=40572191246048d8ff92edee6411
e8766511888d, for GNU/Linux 3.2.0, not stripped
Hello, World!
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 19. Creation and native verification of the statically linked Hello World binary.

```
warn: The se.py script is deprecated. It will be removed in future releases of  gem5.
Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Gr
oup. Legacy stat is deprecated.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:32] into equal-sized buckets. Rounding up.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:8] into equal-sized buckets. Rounding up.
system.remote_gdb: Listening for connections on port 7000
src/sim/power_state.cc:105: warn: PowerState: Already in the requested power state, request ignored
**** REAL SIMULATION ****
src/sim/syscall_emul.cc:86: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall rseq(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall rseq(...)
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
Hello, World!
Hello, World!
Exiting @ tick 125599500 because exiting with last active thread context
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 20. Successful two-thread SMT run showing two Hello World outputs.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ grep -E "simInsts|numCycles|\.ipc|\.cpi" m5out_ilp_smt_static/stats.txt | head -30
simInsts                                    269960                       # Number of instructions simulated (Count)
system.cpu.numCycles                        251200                       # Number of cpu cycles simulated (Cycle)
system.cpu.cpi                            0.930508                       # CPI: cycles per instruction (core level) ((Cycle/Count)
)
system.cpu.ipc                            1.074682                       # IPC: instructions per cycle (core level) ((Count/Cycle)
)
system.cpu.commitStats0.cpi               1.861016                       # CPI: cycles per instruction (thread level) ((Cycle/Coun
t))
system.cpu.commitStats0.ipc               0.537341                       # IPC: instructions per cycle (thread level) ((Count/Cycl
e))
system.cpu.commitStats1.cpi               1.861016                       # CPI: cycles per instruction (thread level) ((Cycle/Coun
t))
system.cpu.commitStats1.ipc               0.537341                       # IPC: instructions per cycle (thread level) ((Count/Cycl
e))
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 21. SMT instruction count, cycles, CPI, total IPC, and per-thread IPC.

A fair single-thread baseline was then run with the same static executable, DerivO3CPU, cache option, and se.py path. This avoided comparing the SMT result against a different binary or configuration.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ rm -rf m5out_ilp_single_static && build/X86/gem5.opt --outdir=m5out_ilp_single_static \
configs/deprecated/example/se.py \
--cpu-type=DerivO3CPU \
--caches \
-c ./hello_static
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 25.1.0.1
gem5 compiled Oct  4 2026 09:48:48
gem5 started Oct 18 2026 12:03:17
gem5 executing on DESKTOP-C34OBRR, pid 1765
command line: build/X86/gem5.opt --outdir=m5out_ilp_single_static configs/deprecated/example/se.py --cpu-type=DerivO3CPU --caches -c
./hello_static

warn: The se.py script is deprecated. It will be removed in future releases of  gem5.
Global frequency set at 1000000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Gr
oup. Legacy stat is deprecated.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:32] into equal-sized buckets. Rounding up.
src/base/stats/storage.hh:278: warn: Bucket size (4) does not divide range [1:8] into equal-sized buckets. Rounding up.
system.remote_gdb: Listening for connections on port 7000
**** REAL SIMULATION ****
src/sim/syscall_emul.cc:86: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:86: warn: ignoring syscall rseq(...)
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
Hello, World!
Exiting @ tick 91624000 because exiting with last active thread context
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 22. Matching single-thread static baseline.

```
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$ grep -E "simInsts|numCycles|\.ipc|\.cpi" m5out_ilp_single_static/stats.txt | head -20
simInsts                                     134980                       # Number of instructions simulated (Count)
system.cpu.numCycles                         183249                       # Number of cpu cycles simulated (Cycle)
system.cpu.cpi                              1.357601                       # CPI: cycles per instruction (core level) ((Cycle/Count)
)
system.cpu.ipc                              0.736593                       # IPC: instructions per cycle (core level) ((Count/Cycle)
)
system.cpu.commitStats0.cpi                 1.357601                       # CPI: cycles per instruction (thread level) ((Cycle/Coun
t))
system.cpu.commitStats0.ipc                 0.736593                       # IPC: instructions per cycle (thread level) ((Count/Cycl
e))
vishnusahithjella@DESKTOP-C34OBRR:~/gem5$
```

Figure 23. Matching single-thread static baseline statistics.

Metric	1 Thread	2-Thread SMT	Interpretation
Total instructions	134,980	269,960	Exactly two copies of the workload in SMT
CPU cycles	183,249	251,200	Two SMT workloads finish in fewer cycles than two sequential runs
Overall CPI	1.357601	0.930508	Lower aggregate cycle cost with SMT
Overall IPC	0.736593	1.074682	45.9% higher overall throughput
Per-thread IPC	0.736593	0.537341 each	Each thread slows, but aggregate throughput increases

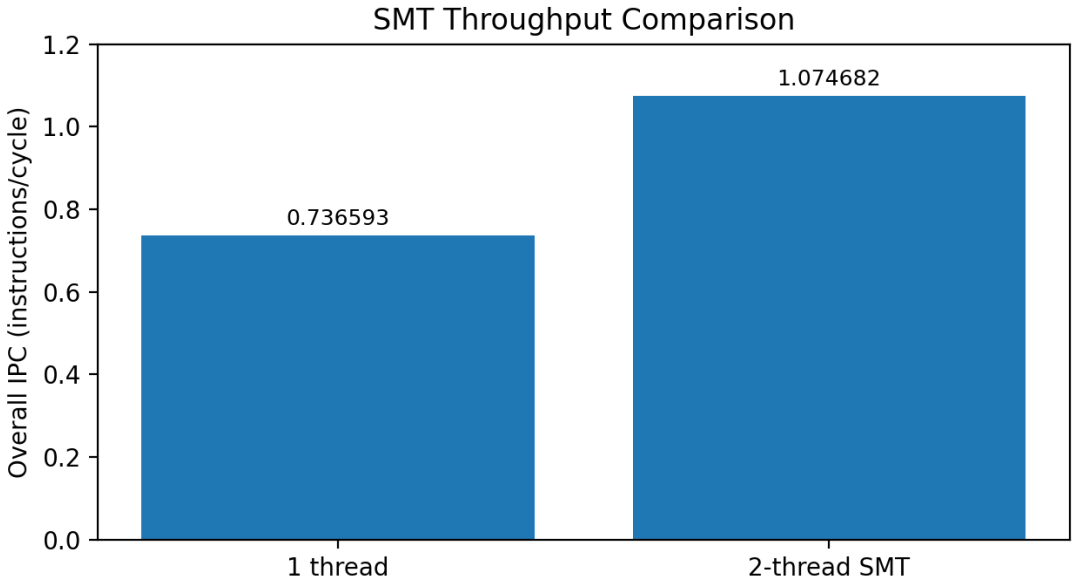


Figure 24. Overall IPC for one thread versus two-thread SMT.

The SMT result demonstrates the distinction between single-thread latency and total throughput. Each SMT thread achieved an IPC of about 0.537, below the 0.737 IPC of the single-thread baseline because the two threads share pipeline and execution resources. However, total processor IPC rose to 1.075, an improvement of approximately 45.9%. Running two copies sequentially would require about 366,498 cycles based on the single-thread measurement, whereas the SMT run completed both in 251,200 cycles. Thus, SMT improved aggregate resource utilization even though it did not double performance.

13. Interaction of ILP Techniques and Observed Limitations

The experiments show that ILP mechanisms interact rather than operate independently. Branch prediction determines whether useful instructions reach the out-of-order window. Issue width controls how many ready instructions may enter execution in a cycle. SMT introduces instructions from another thread when one thread cannot fully use the available resources. These mechanisms can therefore compensate for different causes of pipeline underutilization.

There are also important limitations. The Hello World workload is intentionally small and does not stress floating-point execution, long dependence chains, or large memory working sets. The predictor experiment compares two active predictors rather than a predictor against a fully disabled O3 front end. The report uses IPC and CPI because a direct average instruction-latency statistic was not collected. Finally, the O3 pipeline visualization in Figure 3 is a conceptual stage mapping rather than a saved O3PipeView cycle trace. These limitations are stated explicitly so that the results are not overstated.

Despite those limits, the controlled comparisons answer the main performance questions. Better branch behavior slightly improved IPC, constraining issue width reduced IPC, and SMT substantially increased total throughput. The troubleshooting sequence also showed that realistic processor models have configuration dependencies: a stable ILP experiment requires consistent CPU, system, interrupt, cache, and workload settings.

14. Conclusion

Assignment 4 connected instruction-level parallelism theory with practical gem5 measurements. The literature review showed that modern ILP research is moving toward complexity-effective techniques: better instruction delivery, smarter branch prediction, specialized runahead execution, and lightweight out-of-order scheduling. The gem5 experiments produced the same general message at a smaller scale. LocalBP reduced the measured branch misprediction rate and slightly improved IPC, the normal multi-issue O3 core outperformed the issueWidth = 1 configuration, and two-thread SMT improved aggregate IPC by about 45.9% over the matching single-thread baseline. Together, the theoretical and experimental results demonstrate that processor performance depends on finding independent work while balancing prediction quality, hardware resources, power, and complexity.

References

Chaturvedi, I., Godala, B. R., Wu, Y., Xu, Z., Iliakis, K., Eleftherakis, P. E., Xydis, S., Soudris, D., Sorensen, T., Campanoni, S., Aamodt, T. M., & August, D. I. (2024). GhOST: A GPU out-of-order scheduling technique for stall reduction. In 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA) (pp. 1–16). IEEE. https://doi.org/10.1109/ISCA59077.2024.00011

gem5 Project. (2026). gem5 documentation. https://www.gem5.org/documentation/

Harsh, B. S., & Renau, J. (2026). HOTP - High-confidence-only taken predictor for efficient superscalar multi-level branch predictor. In 2026 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS) (pp. 380–390). IEEE. https://doi.org/10.1109/ISPASS69572.2026.00044

Hennessy, J. L., & Patterson, D. A. (2019). Computer architecture: A quantitative approach (6th ed.). Morgan Kaufmann.

Matsuo, R., Koizumi, T., Irie, H., Sakai, S., & Shioya, R. (2024). TURBULENCE: Complexity-effective out-of-order execution on GPU with distance-based ISA. IEEE Computer Architecture Letters, 23(2), 175–178. https://doi.org/10.1109/LCA.2023.3289317

Naithani, A., Roelandts, J., Ainsworth, S., Jones, T. M., & Eeckhout, L. (2023). Decoupled vector runahead. In 56th IEEE/ACM International Symposium on Microarchitecture (MICRO 2023) (pp. 17–31). ACM. https://doi.org/10.1145/3613424.3614255

Zhang, Y., Khan, T. A., Pokam, G., Kasikci, B., Litz, H., & Devietti, J. (2022). OCOLOS: Online Code Layout Optimization. In 2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO) (pp. 530–545). IEEE. https://doi.org/10.1109/MICRO56248.2022.00045

GitHub Link: https://github.com/sahithpintu/MSCS-531-M40-Computer-Architecture-Design-Assignment-4